

TP Statistiques – PBS2

Paul Minchella

Avril 2026

Table des matières

1	Correction du TP1	2
1.1	Lecture des données	2
1.2	Examen global des données	2
1.2.1	Valeurs manquantes	3
1.2.2	Valeurs aberrantes	3
1.3	Visualisations descriptives	3
1.3.1	Histogrammes et pairplots	3
1.3.2	Boxplots et règle de Tukey	5
1.4	Matrice de corrélation	7
1.5	Visualisation des catégories de véhicules	8
1.6	Régression linéaire multiple	9
1.6.1	Écriture du modèle	9
1.6.2	Interprétation générale du summary	10
1.6.3	Retrouver les coefficients par la formule matricielle	10
1.6.4	Prédictions et comparaison avec les vraies valeurs	10
1.6.5	Étude des résidus et normalité	11
1.6.6	Individus aux paramètres plausibles	13
1.7	Imprécision des coefficients	13
1.8	Caractère significatif des coefficients et choix d'un modèle réduit	13
1.9	Train-Test Split	15
1.9.1	Présentation de la procédure	15
1.9.2	Application pour ce TP	15
1.10	Conclusion générale TP1	18
	Synthèse TP1	19

1 Correction du TP1

Dans ce TP, on dispose d'un jeu de données décrivant un ensemble de véhicules. Une étape absolument indispensable, avant toute modélisation, consiste à effectuer une **analyse descriptive complète** des données. En effet, avant de construire un modèle, il faut d'abord comprendre le dataset que l'on va manipuler : nombre d'observations, nature des variables, valeurs manquantes, éventuelles valeurs atypiques, liens entre variables, etc.

Notre base d'apprentissage est constituée de **18 véhicules** décrits par **8 variables**. On peut imaginer le scénario suivant : un garage ou un concessionnaire souhaite étudier la relation entre le **prix** d'un véhicule et ses autres caractéristiques mesurées. L'objectif final est qu'à l'arrivée d'un nouveau véhicule, on puisse estimer son prix à partir de ses covariables, autrement dit construire un modèle permettant d'**inférer** ou d'**estimer** le prix.

1.1 Lecture des données

On commence par se placer dans le bon répertoire de travail, puis par lire le fichier de données.

```
setwd("~/Ton/Folder/approprié")
df <- read.csv("Vehicules_TD_2023.csv",
               sep = ";", dec = ".", header = TRUE,
               stringsAsFactors = FALSE)
```

La commande `setwd()` signifie *set working directory* : elle fixe le **répertoire de travail courant**. Concrètement, cela indique à R dans quel dossier il doit chercher les fichiers que l'on va lire ou écrire. C'est une bonne pratique car cela évite d'avoir à préciser à chaque fois le chemin complet vers un fichier, et cela rend le script plus lisible.

La commande `read.csv()` permet de lire le fichier `Vehicules_TD_2023.csv`. Les arguments ont ici une importance particulière :

- `sep = ";"` signifie que les colonnes du fichier sont séparées par des points-virgules ;
- `dec = "."` signifie que le séparateur décimal utilisé est le point ;
- `header = TRUE` signifie que la première ligne du fichier contient les noms des variables ;
- `stringsAsFactors = FALSE` empêche R de convertir automatiquement les chaînes de caractères en facteurs.

Une fois les données importées, les premières commandes à effectuer sont des commandes de description globale :

```
n_obs <- nrow(df); n_obs
n_var <- ncol(df); n_var
str(df)
summary(df)
sapply(df[, sapply(df, is.numeric)], sd, na.rm = TRUE)
```

La commande `nrow(df)` renvoie le nombre d'observations, tandis que `ncol(df)` renvoie le nombre de variables. La fonction `str(df)` fournit la structure du tableau de données : noms des variables, types (`numeric`, `character`, etc.), aperçu de leur contenu. Ensuite, `summary(df)` donne des statistiques descriptives élémentaires variable par variable : minimum, maximum, médiane, quartiles et moyenne pour les variables numériques. Enfin, la dernière ligne affiche le calcul des écart-types par variable aléatoire **numérique**. *Cette grandeur est **indispensable** pour toute étude descriptive un peu sérieuse.*

1.2 Examen global des données

Une bonne pratique consiste à examiner immédiatement le nombre d'observations et le nombre de covariables numériques mobilisables pour la modélisation.

```
p <- ncol(df) - 1 # On retire la variable cible du comptage
```

Ici, on travaille avec $n = 18$ observations, pour $p = 7$ variables explicatives ($p_{\text{num}} = 6$ en réalité, on ne travaille pas avec la GAMME). Le jeu de données est de petite taille, ce qui devra être gardé à l'esprit dans toute l'analyse : certains diagnostics seront peu puissants statistiquement.

1.2.1 Valeurs manquantes

On vérifie ensuite la présence éventuelle de valeurs manquantes :

```
colSums(is.na(df))
sum(is.na(df))
df[!complete.cases(df), ]
```

- `colSums(is.na(df))` indique, pour chaque variable, le nombre de valeurs manquantes ;
- `sum(is.na(df))` donne le nombre total de valeurs manquantes dans le tableau ;
- `df[!complete.cases(df), ]` affiche les lignes incomplètes.

Dans notre cas, on ne détecte pas de valeurs manquantes. C'est une bonne nouvelle, car cela évite d'avoir à traiter un problème d'imputation ou de suppression d'observations.

1.2.2 Valeurs aberrantes

À ce stade, il est encore un peu tôt pour conclure à l'existence de valeurs aberrantes. Une simple lecture du tableau peut toutefois faire émerger des soupçons. Par exemple, la variable `CYL` de l'observation 13 peut paraître atypique. Néanmoins, on ne peut pas trancher sérieusement sans visualisation, d'où un examen descriptif par plusieurs graphiques (boxplots, histogrammes, ...).

1.3 Visualisations descriptives

1.3.1 Histogrammes et pairplots

On commence par isoler les variables numériques :

```
num_cols <- names(df)[sapply(df, is.numeric)]
num_cols
```

On peut représenter les **histogrammes** des variables numériques, pour visualiser leur distribution.

```
# Variables explicatives
vars_features <- setdiff(num_cols, "PRIX")
par(mfrow = c(2, 3))
for (v in vars_features) {
  hist(df[[v]], main = paste("Histogramme de", v),
       xlab = v, col = "lightblue")
}

# Variable à modéliser
par(mfrow = c(1, 1))
hist(df$PRIX, main = "Histogramme du PRIX", xlab = "PRIX", col = "#735EF1")
```

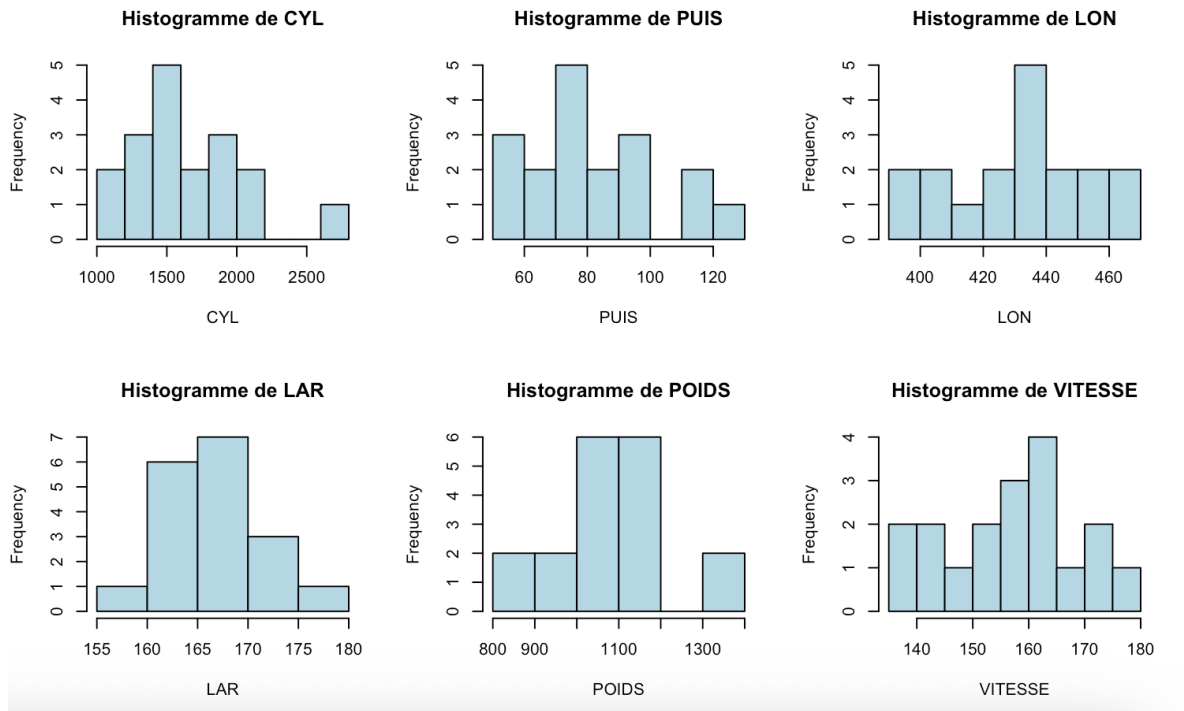


FIGURE 1 – Histogrammes des variables explicatives du modèle (CYL, PUIS, LON, LAR, POIDS, VITESSE). Ces représentations permettent d'examiner la forme des distributions, de repérer d'éventuelles valeurs atypiques et d'évaluer qualitativement la dispersion des covariables avant la phase de modélisation.

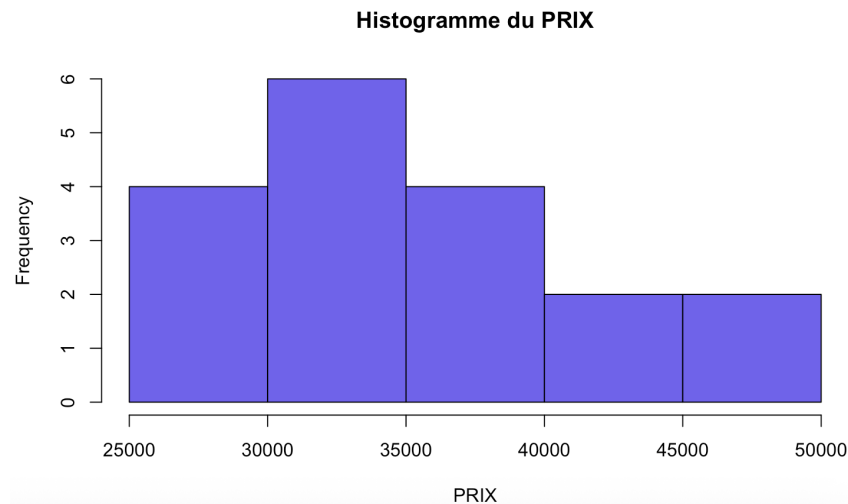


FIGURE 2 – Histogramme de la variable réponse PRIX. Cette représentation permet d'évaluer la distribution globale des prix observés dans l'échantillon et d'identifier une éventuelle asymétrie ou dispersion importante.

Ensuite, une autre figure intéressante est le *pairplot*, pour observer les relations deux à deux entre variables via chacun des *scatterplots*.

```
pairs(df[, num_cols], main = "Pairplot")
```

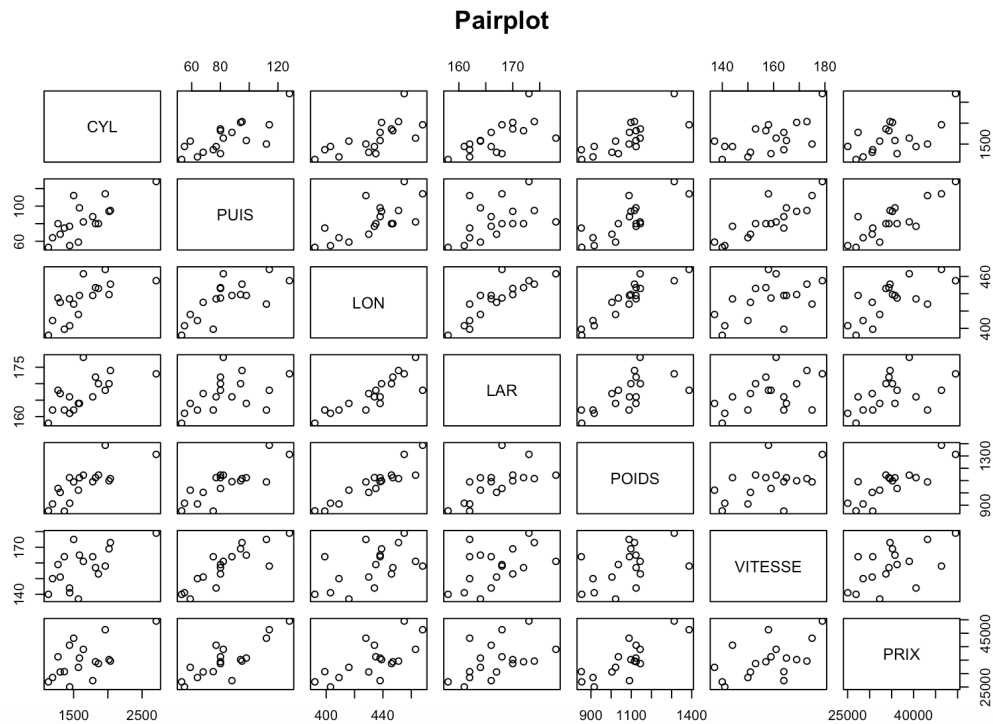


FIGURE 3 – Pairplot entre les variables numériques

Le *pairplot* permet de voir rapidement si certaines relations semblent linéaires, croissantes, décroissantes ou au contraire très diffuses. C'est une étape importante avant la régression, car elle donne une première intuition sur la pertinence d'un modèle linéaire.

1.3.2 Boxplots et règle de Tukey

On examine également les variables au moyen de **boxplots** :

```
par(mfrow = c(3, 2))
for (v in vars_features) {
  boxplot(df[[v]], main = paste("Boxplot de", v),
    horizontal = TRUE, col='lightblue')
}

par(mfrow = c(1, 1))
boxplot(df$PRIX, main = "Boxplot du PRIX",
  horizontal = TRUE, col = "#735EF1")
```

Un **boxplot** résume graphiquement la distribution d'une variable :

- la boîte va du premier quartile Q_1 au troisième quartile Q_3 ;
- le trait dans la boîte correspond à la médiane ;
- les moustaches s'étendent vers les valeurs extrêmes non aberrantes ;
- les points isolés au-delà des moustaches sont des valeurs potentiellement atypiques.

Le critère usuel est la **règle de Tukey** : une observation est souvent considérée comme atypique si elle se situe en dehors de l'intervalle

$$[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}],$$

où $\text{IQR} = Q_3 - Q_1$ est l'écart interquartile.

Ici, les boxplots suggèrent qu'une valeur de la variable **CYL** peut effectivement apparaître atypique. Idem pour deux observations au vu du boxplot du **POIDS**. Néanmoins, compte tenu de la petite taille de l'échantillon, il convient de rester prudent : une valeur extrême n'est pas forcément une erreur de saisie, elle peut simplement refléter un véhicule réellement différent des autres.

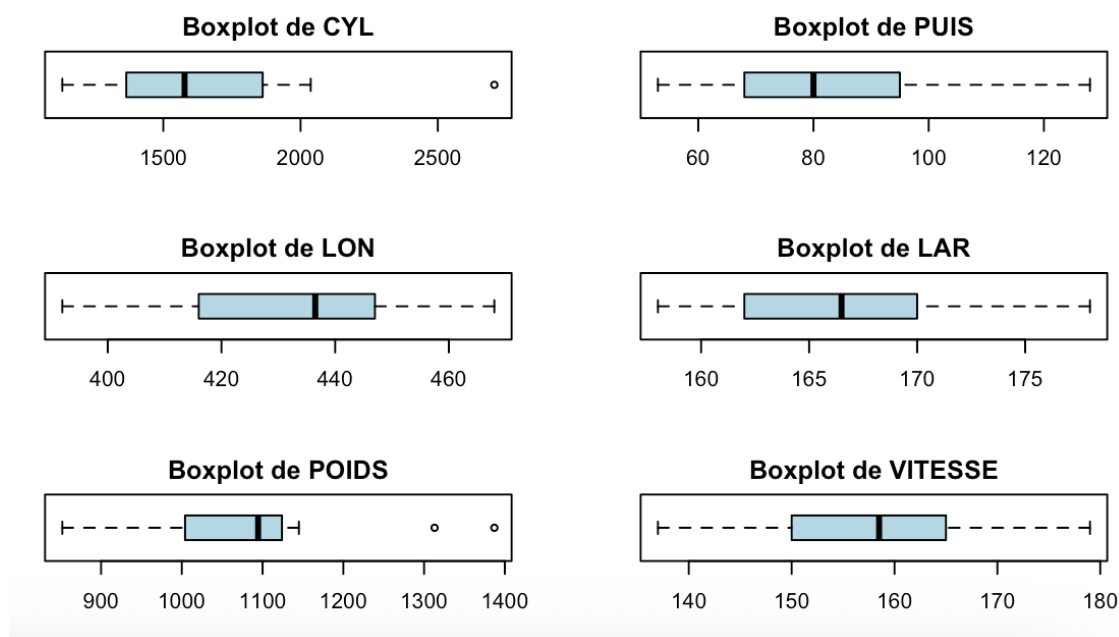


FIGURE 4 – Boxplots des variables explicatives numériques (**CYL**, **PUIS**, **LON**, **LAR**, **POIDS**, **VITESSE**). Ces représentations permettent d'analyser la dispersion des variables, leur asymétrie éventuelle ainsi que la présence de valeurs atypiques.

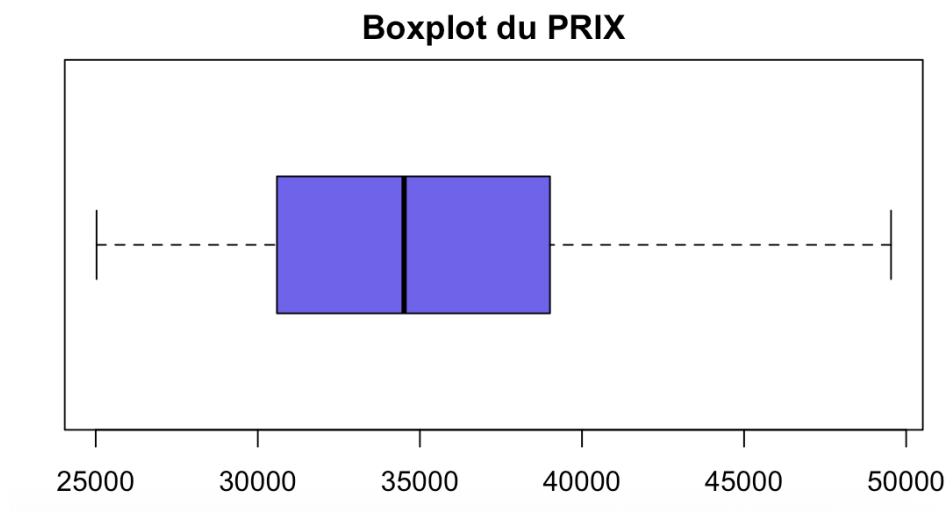


FIGURE 5 – Boxplot de la variable réponse **PRIX**. Il permet d'identifier la dispersion des prix des véhicules ainsi que l'absence apparente de valeurs aberrantes marquées dans l'échantillon.

1.4 Matrice de corrélation

On cherche ensuite à quantifier les relations linéaires entre les variables numériques.

Définition 1.1 (Matrice de corrélation). Soient $X^{(1)}, \dots, X^{(p)}$ des variables numériques. Chaque variable $X^{(j)}$ a pour écart-type $\sigma_{X^{(j)}}$. La *matrice de corrélation* est la matrice carrée **symétrique**

$$\text{Corr}(X) = (\rho_{jk})_{1 \leq j, k \leq p},$$

où

$$\rho_{jk} = \text{Corr}(X^{(j)}, X^{(k)}) = \frac{\text{Cov}(X^{(j)}, X^{(k)})}{\sigma_{X^{(j)}} \sigma_{X^{(k)}}}.$$

Chaque coefficient ρ_{jk} est compris entre -1 et 1 .

Une corrélation proche de 1 indique une forte liaison linéaire positive, une corrélation proche de -1 une forte liaison linéaire négative, et une corrélation proche de 0 l'absence de liaison linéaire marquée.

On commence par calculer la matrice de corrélation empirique entre les variables numériques :

```
all_corr <- cor(df[num_cols])
all_corr
```

Ce qui renvoie :

	CYL	PUIS	LON	LAR	POIDS	VITESSE	PRIX
CYL	1.0000000	0.7496052	0.6937336	0.6532935	0.7750733	0.6171567	0.6061196
PUIS	0.7496052	1.0000000	0.7103380	0.4546125	0.8109449	0.8495398	0.8389992
LON	0.6937336	0.7103380	1.0000000	0.8539106	0.9158759	0.5238072	0.6971867
LAR	0.6532935	0.4546125	0.8539106	1.0000000	0.6462145	0.4714780	0.4842995
POIDS	0.7750733	0.8109449	0.9158759	0.6462145	1.0000000	0.4719416	0.8275308
VITESSE	0.6171567	0.8495398	0.5238072	0.4714780	0.4719416	1.0000000	0.5466699
PRIX	0.6061196	0.8389992	0.6971867	0.4842995	0.8275308	0.5466699	1.0000000

Cette matrice contient l'ensemble des coefficients de corrélation linéaire deux à deux entre les variables. Elle permet déjà d'identifier certaines relations fortes : par exemple, la puissance (PUIS) est fortement corrélée au prix (PRIX), tout comme le poids (POIDS), tandis que d'autres variables présentent des corrélations plus modérées.

Cependant, une lecture directe de cette matrice numérique reste peu intuitive dès que le nombre de variables augmente. Il est donc souvent préférable de représenter cette information sous une forme *visuelle*, qui permet d'identifier immédiatement les corrélations fortes (positives ou négatives) et les groupes de variables liées entre elles.

Sous R, on peut ainsi visualiser la matrice de corrélation de manière synthétique (*et bien plus esthétique...*) à l'aide du package `corrplot` :

```
library(corrplot)
all_corr <- cor(df[, num_cols])
corrplot(all_corr)
```

Le rendu est en Figure 6.

Cette matrice permet d'identifier rapidement des groupes de variables fortement corrélées, ce qui est très utile avant de lancer une régression multiple. En effet, des variables très corrélées entre elles peuvent poser des problèmes d'interprétation des coefficients.

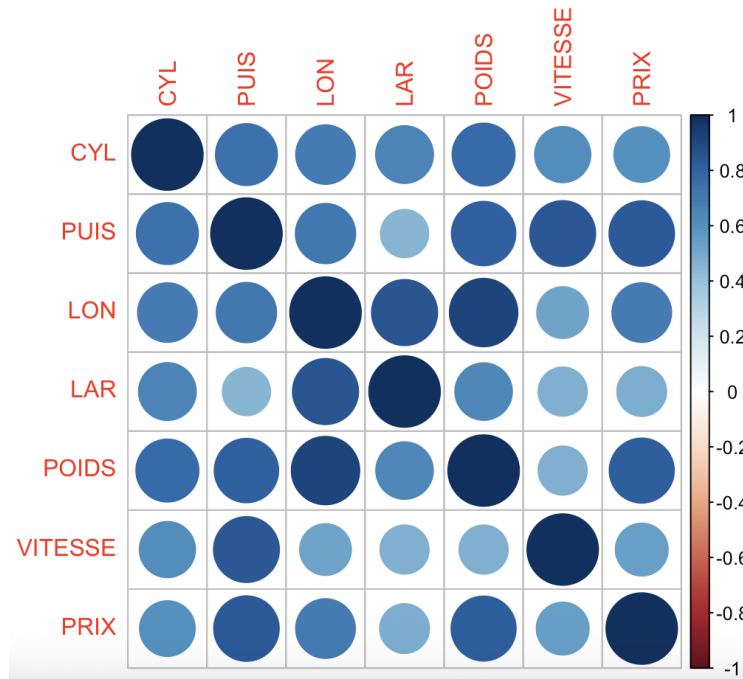


FIGURE 6 – Matrice de corrélation des variables numériques du jeu de données véhicules.

1.5 Visualisation des catégories de véhicules

Il est naturel, compte tenu de la présence d'une variable en classe, de se demander si les trois catégories de véhicules sont visuellement séparables. Si l'on projette les véhicules dans certains plans définis par deux variables numériques, observe-t-on des groupes distincts ?

Cette question est importante car elle renseigne déjà sur la structure du dataset. Une séparation visuelle nette entre catégories suggère que certaines variables portent une information discriminante forte. On peut représenter par exemple les véhicules dans les plans (CYL, PUIS) et (POIDS, PUIS), en coloriant selon la catégorie et en ajoutant les centres de gravité des groupes (cf le script R qui définit la méthode `plot_plan`).

Ces graphiques permettent d'apprécier visuellement si les catégories sont bien distinctes ou au contraire fortement mélangées. Une telle étape est très utile avant modélisation, car elle fournit une intuition concrète sur la structure géométrique des données.

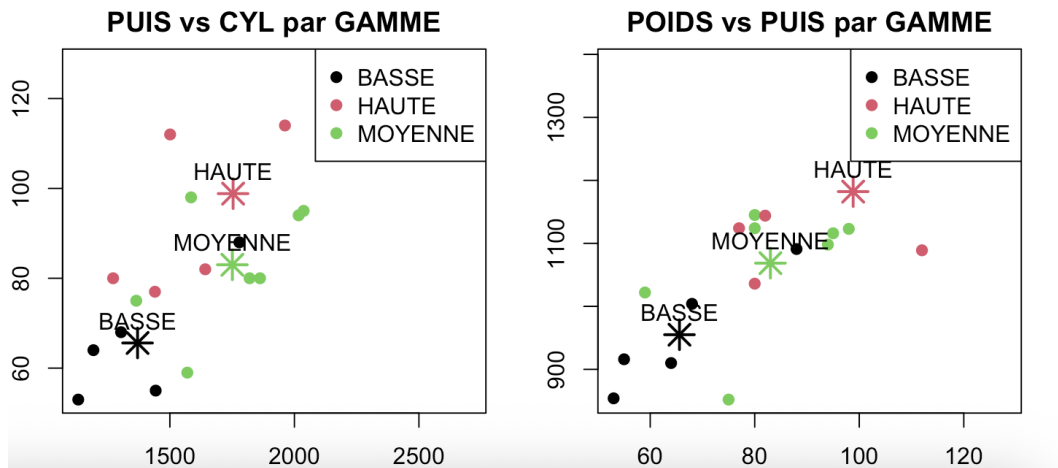


FIGURE 7 – Représentation des véhicules dans les plans (POIDS, CYL) et (POIDS, PUIS) avec centres de gravité par gamme

Interprétation des centres de gravité. Chaque astérisque représente le *centre de gravité* (ou barycentre) des observations appartenant à une même catégorie de la variable **GAMME**. Il correspond à la moyenne des coordonnées des véhicules dans le plan considéré. Dans le plan (PUIS, CYL), on observe une structuration claire :

- la gamme **BASSE** est associée à de faibles valeurs de cylindrée et de puissance ;
- la gamme **MOYENNE** occupe une position intermédiaire ;
- la gamme **HAUTE** correspond aux valeurs les plus élevées.

Le même phénomène apparaît dans le plan (POIDS, PUIS) : les centres de gravité sont ordonnés selon un gradient croissant de puissance et de poids. Cela indique que ces variables contribuent fortement à différencier les catégories de véhicules.

Ainsi, ces représentations suggèrent que les variables **CYL**, **PUIS** et **POIDS** possèdent un pouvoir discriminant exploitable (*sans dire absolu non plus...*) pour expliquer la variable qualitative **GAMME**.

1.6 Régression linéaire multiple

Nous cherchons désormais à modéliser le **prix** en fonction des autres variables numériques.

1.6.1 Écriture du modèle

Le modèle de régression linéaire multiple s'écrit, pour toute observation $i = 1, \dots, n$:

$$Y_i = \beta_0 + \sum_{j=1}^p \beta_j X_i^{(j)} + \varepsilon_i,$$

où :

- Y_i est ici le prix du i -ième véhicule ;
- $X_i^{(j)}$ désigne la j -ième covariable du véhicule i ;
- $\beta_0, \beta_1, \dots, \beta_p$ sont les paramètres inconnus ;
- ε_i est le terme d'erreur.

Sous R, cela s'encode comme tel :

```
Modele <- lm(PRIX ~ CYL + PUIS + LON + LAR + POIDS + VITESSE, data = df)
```

L'ensemble des informations principales sur le modèle ajusté est obtenu avec :

```
summary(Modele)
```

On obtient en sortie :

```
Call:
lm(formula = formule, data = df)

Residuals:
    Min       1Q   Median       3Q      Max
-725.52 -429.80  -38.65  476.44  827.83

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -34993.19   11442.06  -3.058   0.011 *
CYL           -13.55     3.96   -3.421   0.006 **
PUIS           907.99    320.43   2.834   0.016 *
LON          -556.44    234.06  -2.377   0.037 *
LAR          2128.52    142.85  14.900   0.000 ***
POIDS         18.46     60.69   0.304   0.767
VITESSE      -746.73    401.95  -1.858   0.090 .
```

```

Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 628.8 on 11 degrees of freedom
Multiple R-squared: 0.9943, Adjusted R-squared: 0.9912
F-statistic: 321.6 on 6 and 11 DF, p-value: 1.063e-11

```

1.6.2 Interprétation générale du summary

La colonne **Estimate** donne les coefficients estimés $\hat{\beta}_j$. La colonne **Std. Error** donne leur écart-type estimé. La colonne **t value** donne la statistique de Student associée au test

$$H_0 : \beta_j = 0.$$

Enfin, la colonne **Pr(>|t|)** donne la p-value de ce test.

- On observe par exemple que les variables **CYL**, **PUIS**, **LON** et surtout **LAR** apparaissent significatives aux seuils usuels, tandis que **POIDS** ne semble pas l'être dans ce modèle.
- Le **Residual standard error** donne $\hat{\sigma} = 628.8$. Il s'agit d'une estimation de l'écart-type du bruit résiduel.
- Le coefficient de détermination multiple vaut ici $R^2 = 0.9943$, ce qui signifie que le modèle explique environ 99.43% de la variabilité observée du prix dans l'échantillon. Le R^2 ajusté vaut 0.9912, ce qui reste très élevé. Le modèle semble donc très performant sur ces données.
- Enfin, le test global de Fisher donne une p-value extrêmement petite : 1.063×10^{-11} . On rejette donc très nettement l'hypothèse selon laquelle tous les coefficients explicatifs seraient nuls. Le modèle est donc **globalement significatif**.

1.6.3 Retrouver les coefficients par la formule matricielle

Le vecteur des moindres carrés s'écrit

$$\hat{\beta} = \left(\mathbb{X}^\top \mathbb{X} \right)^{-1} \mathbb{X}^\top \mathbf{Y}.$$

On peut vérifier sous R que l'on retrouve exactement les mêmes coefficients :

```

X <- model.matrix(Modele)
Y <- df$PRIX

beta_chap <- solve(t(X) %*% X) %*% t(X) %*% Y
beta_chap

```

Cette vérification est importante (*au cas où l'on douterait de R...*) : elle confirme que la fonction `lm()` implémente bien la solution théorique du problème des moindres carrés.

1.6.4 Prédiction et comparaison avec les vraies valeurs

Une fois le modèle ajusté, on peut calculer les valeurs prédites $\hat{Y}_i$. On les compare ensuite aux valeurs réellement observées Y_i .

```

y_hat <- fitted(Modele)

plot(Y, y_hat,
     xlab = "PRIX observé",
     ylab = "PRIX prévu",
     pch = 19,
     main = "PRIX observé vs PRIX prévu")
abline(0, 1, col = "red", lwd = 2)

```

On pourra insérer la figure correspondante :

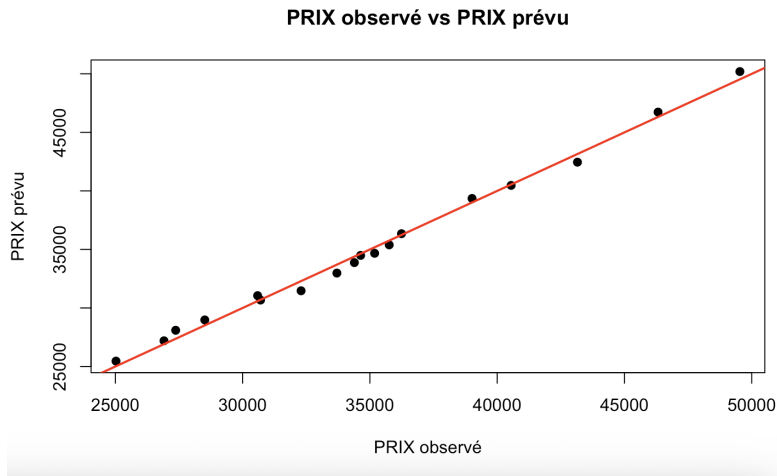


FIGURE 8 – Prix observés versus prix prédits par le modèle.

L'idée géométrique est simple : si le modèle ajustait parfaitement les données, tous les points seraient exactement alignés sur la première bissectrice d'équation $y = x$. Ici, les points sont très proches de cette droite, ce qui confirme visuellement la très bonne qualité d'ajustement déjà suggérée par le R^2 .

1.6.5 Étude des résidus et normalité

Les résidus mesurent l'écart entre la vraie valeur et la valeur ajustée/estimée :

$$\hat{\varepsilon}_i = Y_i - \hat{Y}_i.$$

Sous R, on les obtient avec :

```
res <- residuals(Modele)
```

On peut vérifier sur un exemple que `res` coïncide bien avec $Y - \hat{Y}$.

```
head(Y - y_hat)
head(res)
```

Pour examiner la normalité des résidus, on trace un histogramme ainsi qu'un QQ-plot :

```
par(mfrow = c(1, 2))
hist(res, breaks = 15, main = "Histogramme des résidus",
     xlab = "Résidus", col = "purple")
qqnorm(res)
qqline(res, col = "red", lwd = 2)
par(mfrow = c(1, 1))
```

On se doit d'avoir le réflexe d'établir ce type de graphes mais l'interprétation de la Figure 9 doit rester prudente ici car l'échantillon est très petit ($n = 18$). L'histogramme est ici peu convaincant à lui seul, précisément parce qu'il y a trop peu de données pour faire émerger une forme stable. Le QQ-plot, en revanche, donne une information plus utile : les points ne sont pas parfaitement alignés sur la droite de référence, mais l'écart ne paraît pas non plus extrêmement marqué.

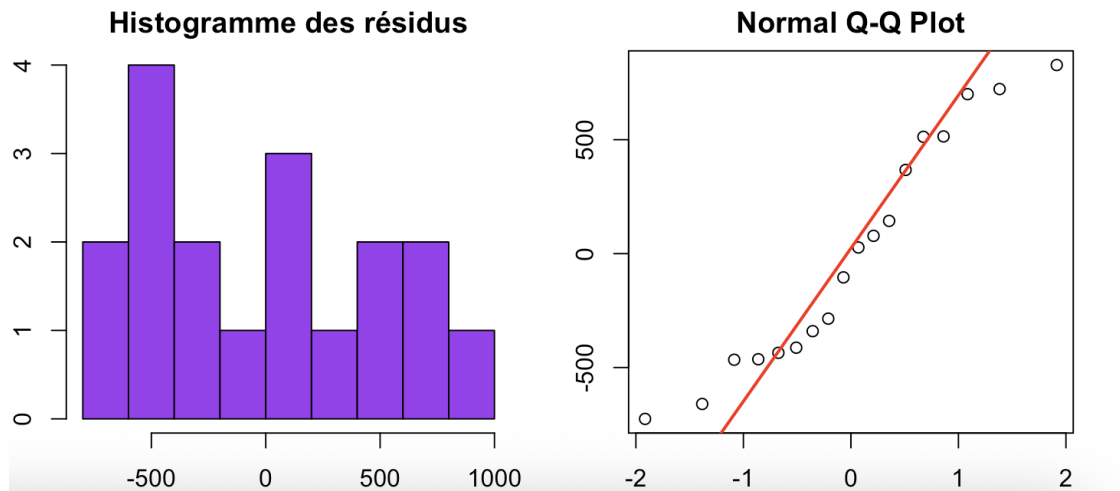


FIGURE 9 – Histogramme et QQ-plot des résidus.

On peut compléter cette analyse visuelle par un test de Shapiro–Wilk, qui a pour hypothèse nulle

$$H_0 : \varepsilon \sim \mathcal{N} \iff \varepsilon \text{ suit une loi normale.}$$

Il est directement implémenté dans R via la commande :

```
shapiro.test(res)
```

On obtient comme résultat :

```
Shapiro-Wilk normality test
data:  res
W = 0.92802, p-value = 0.1792
```

La p-value vaut ici 0.1792, ce qui est supérieure à 0.05. On ne rejette donc pas l’hypothèse de normalité des résidus au seuil de 5%. Il faut néanmoins interpréter ce résultat avec prudence : *ne pas rejeter* la normalité ne signifie pas que les résidus sont certainement gaussiens ; cela signifie simplement que, compte tenu de la faible taille de l’échantillon, on ne dispose pas d’éléments statistiques suffisants pour conclure à une non-normalité.

Centralité des résidus. Dans le modèle de régression linéaire, une hypothèse importante est que les résidus sont *centrés*, c’est-à-dire de moyenne nulle :

$$\mathbb{E}(\varepsilon_i) = 0.$$

Même si cette propriété est en partie garantie par la méthode des moindres carrés, il est utile de la vérifier empiriquement sur les données. Pour cela, on réalise un test de Student grâce à la commande :

```
t.test(res)
```

Ce test renvoie :

```
One Sample t-test
data:  res
t = -5.0995e-16, df = 17, p-value = 1
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
```

```
-251.5472 251.5472
sample estimates:
mean of x
-6.079974e-14
```

La p-value vaut ici 1, ce qui signifie que l'on ne rejette pas l'hypothèse nulle. En gros, rien ne permet de conclure que la moyenne des résidus diffère de zéro. L'hypothèse de centralité des résidus est donc compatible avec les données observées.

1.6.6 Individus aux paramètres plausibles

Je n'ai aucune idée de ce qui est voulu ici et de ce que le bro attend de nous. J'imagine qu'il faut choisir soi-même les sept covariables *plausibles* et voir comment le modèle infère le prix...

1.7 Imprécision des coefficients

L'objectif de cette question est de mettre en évidence une propriété importante des modèles de régression linéaire : lorsque l'échantillon est de petite taille, les coefficients estimés peuvent être sensibles aux observations individuelles. Le principe de l'expérience pour cette question est le suivant :

1. on ajuste d'abord le modèle linéaire sur l'ensemble complet des données ;
2. puis on supprime une observation ;
3. on ajuste à nouveau le modèle ;
4. on compare les nouveaux coefficients obtenus avec les coefficients initiaux.

Cette procédure permet de mesurer la *stabilité* des estimateurs des moindres carrés. On obtient par exemple les résultats suivants :

	Intercept	CYL	PUIS	LON	LAR	VITESSE
Modèle complet	-32085.08	-12.37	1004.83	-485.86	2101.96	-868.43
Sans ligne 1	-31696.30	-12.44	1004.31	-486.57	2101.53	-867.25
Sans ligne 5	-37432.29	-12.77	1012.04	-492.10	2157.21	-875.23
Sans ligne 10	-30794.26	-12.38	1006.89	-489.14	2105.34	-872.05

On observe que la suppression d'une seule observation peut modifier sensiblement certaines estimations des coefficients, en particulier l'intercept et, dans une moindre mesure, les coefficients associés aux variables explicatives. Ce phénomène s'explique par deux raisons principales :

- la taille de l'échantillon est relativement faible ;
- certaines observations peuvent être influentes (leur position dans l'espace des covariables est atypique).

Ainsi, dans un petit jeu de données, chaque observation contient une quantité d'information importante et peut affecter l'estimation globale du modèle. Cela justifie l'intérêt d'augmenter la taille de l'échantillon lorsque cela est possible.

1.8 Caractère significatif des coefficients et choix d'un modèle réduit

Nous examinons maintenant le caractère significatif des coefficients du modèle obtenu avec la commande

```
summary(Modele)
```

La sortie fournit, pour chaque variable explicative :

- l'estimation du coefficient (**Estimate**) ;
- son écart-type estimé (**Std. Error**) ;
- la statistique de test de Student (**t value**) ;
- la p-value associée au test

$$H_0 : \beta_j = 0.$$

L'objectif est d'identifier quelles variables expliquent réellement la variable réponse **PRIX**.

Analyse des p-values. On adopte en général un seuil de significativité de 5%.

- La variable LAR est très fortement significative ($p < 10^{-7}$). Elle joue un rôle majeur dans l'explication du prix.
- Les variables CYL, PUIS et LON sont significatives au seuil 5%.
- La variable VITESSE est marginalement significative (seuil 10%).
- La variable POIDS n'est pas significative ($p = 0.767$). Elle ne semble pas apporter d'information supplémentaire une fois les autres variables prises en compte.

Ainsi, toutes les variables sauf POIDS contribuent de manière notable à l'explication du prix dans ce modèle.

Analyse via les intervalles de confiance. On peut confirmer cette analyse en examinant les intervalles de confiance à 95% :

```
confint(Modele)
```

On obtient notamment :

$$IC_{95\%}(\beta_{POIDS}) = [-115.11, 152.03].$$

Comme cet intervalle contient 0, on ne peut pas conclure que le coefficient associé à POIDS est significativement différent de zéro.

En revanche :

$$IC_{95\%}(\beta_{CYL}), \quad IC_{95\%}(\beta_{PUIS}), \quad IC_{95\%}(\beta_{LON}), \quad IC_{95\%}(\beta_{LAR})$$

ne contiennent pas 0, ce qui confirme leur significativité statistique.

Construction d'un modèle réduit. Une démarche classique consiste à simplifier le modèle en retirant les variables non significatives afin de conserver un modèle plus simple, plus interprétable (*au sens des variables explicatives choisies*) et souvent plus robuste. On propose donc un modèle réduit sans la variable POIDS, ne conservant ainsi dans l'étude uniquement les variables significatives :

$$Y_i = \beta_0 + \beta_1 CYL_i + \beta_2 PUIS_i + \beta_3 LON_i + \beta_4 LAR_i + \beta_5 VITESSE_i + \varepsilon_i.$$

Sous R, ce modèle s'écrit :

```
Modele_reduit <- lm(PRIX ~ CYL + PUIS + LON + LAR + VITESSE, data = df)
summary(Modele_reduit)
```

On peut ensuite comparer ce modèle réduit au modèle initial en observant (*entre-autre*) les nouvelles p-values, la valeur de R^2_{adj} , la significativité globale du modèle (test de Fisher), et le comportement des résidus.

Si la qualité d'ajustement reste comparable, alors le modèle réduit est généralement préférable car il est plus parcimonieux.

	Modèle complet	Modèle réduit
Variable non significative	POIDS	Aucune
Erreur standard résiduelle	628.8	604.6
Degrés de liberté	11	12
R^2	0.9943	0.9943
R^2_{adj}	0.9912	0.9919
Statistique de Fisher	321.6	417.5
p-value (test global)	1.063×10^{-11}	5.074×10^{-13}

TABLE 1 – Comparaison entre le modèle complet et le modèle réduit

On observe que la suppression de la variable POIDS, non significative dans le modèle complet, n'entraîne aucune perte de qualité d'ajustement : le coefficient de détermination R^2 reste inchangé, tandis que le R^2_{adj} augmente légèrement.

De plus, l'erreur standard résiduelle diminue et la statistique de Fisher augmente, ce qui indique une amélioration globale de la qualité du modèle.

Le modèle réduit apparaît donc préférable : il est plus parcimonieux, plus interprétable, et au moins aussi performant que le modèle complet du point de vue statistique.

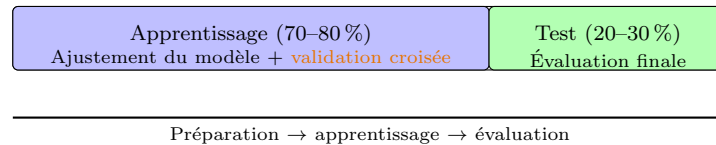
1.9 Train-Test Split

1.9.1 Présentation de la procédure

Pour évaluer correctement un modèle et limiter le sur-apprentissage, il est indispensable de séparer les données en deux ensembles disjoints : un *jeu d'apprentissage* et un *jeu de test*. Le premier sert à estimer les paramètres du modèle, le second à mesurer sa performance sur des données non utilisées pendant l'entraînement. Cette séparation évite une erreur classique : juger un modèle sur les données qu'il a déjà vues conduit presque toujours à une estimation trop optimiste de ses capacités prédictives. En pratique, le découpage s'écrit

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset.$$

$\mathcal{D}$ (données complètes)



Pour un modèle f (régression linéaire), et une perte associée $\mathcal{L}$ (écart quadratique), on calcule ainsi une erreur d'apprentissage

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(x_i), y_i),$$

et une erreur de test

$$\hat{R}_{\text{test}}(\theta) = \frac{1}{n_{\text{test}}} \sum_{i \in \mathcal{D}_{\text{test}}} \mathcal{L}(f(x_i), y_i).$$

Un modèle qui généralise correctement présente des performances comparables sur ces deux ensembles. Un écart marqué, avec une erreur d'apprentissage très faible mais une erreur de test élevée, révèle un sur-apprentissage.

1.9.2 Application pour ce TP

Construction des ensembles train et test sous R. On peut par exemple réserver environ 70% des observations pour l'apprentissage et 30% pour le test.

```
set.seed(777) # Garantir la reproductibilité à chaque relance du code

n <- nrow(df)
id_train <- sample(1:n, size = round(0.7*n))

df_train <- df[id_train, ]
df_test <- df[-id_train, ]
```

On ajuste ensuite le modèle sur l'ensemble d'apprentissage uniquement :

```
Modele_train <- lm(PRIX ~ CYL + PUIS + LON + LAR + VITESSE,
  data = df_train)
summary(Modele_train)
```

Puis on calcule les prédictions sur l'ensemble de test :

```
y_test <- df_test$PRIX
y_pred <- predict(Modele_train, newdata = df_test)
```

Remarque : la commande `predict` est super **importante** car elle permet de renvoyer les valeurs ajustées de toute observation par le modèle appris.

Erreur quadratique sur l'ensemble de test. On évalue la qualité des prédictions à l'aide de l'erreur quadratique moyenne :

$$\text{MSE} = \frac{1}{n_{\text{test}}} \sum_{i \in \mathcal{D}_{\text{test}}} (Y_i - \hat{Y}_i)^2.$$

Sous R :

```
mean((y_test - y_pred)^2)
```

Cette quantité mesure l'écart moyen entre les valeurs observées et les valeurs prédites sur des données non utilisées pour l'apprentissage.

Calcul d'un coefficient R^2 sur l'ensemble de test. On exécute :

```
summary(Modele_train)
```

Le coefficient R^2 affiché par la commande précédente est calculé uniquement sur l'ensemble d'apprentissage. Il mesure donc la qualité d'ajustement du modèle sur les données ayant servi à estimer les coefficients. Pour évaluer la capacité prédictive du modèle, il est plus pertinent de calculer un coefficient analogue sur l'ensemble de test :

$$R_{\text{test}}^2 = 1 - \frac{\sum_{i \in \mathcal{D}_{\text{test}}} (Y_i - \hat{Y}_i)^2}{\sum_{i \in \mathcal{D}_{\text{test}}} (Y_i - \bar{Y}_{\text{test}})^2}.$$

Sous R, on peut calculer ce coefficient par :

```
R2_test <- 1 - sum((y_test - y_pred)^2) /
           sum((y_test - mean(y_test))^2)
```

Ce coefficient mesure la proportion de variance expliquée par le modèle sur des observations nouvelles. Il constitue donc un indicateur plus pertinent de la capacité de généralisation du modèle que le R^2 calculé sur l'ensemble d'apprentissage. En revanche, l'erreur quadratique moyenne calculée précédemment mesure la qualité prédictive sur des données nouvelles.

Un modèle peut présenter un R^2 très élevé sur l'ensemble d'apprentissage tout en ayant une erreur quadratique importante sur l'ensemble de test. Cela indique généralement un sur-ajustement.

Tant qu'à faire, étudier comment se répartissent les valeurs prédites par rapport aux valeurs réelles. Après estimation du modèle sur l'ensemble d'apprentissage, on calcule les prédictions sur l'ensemble de test :

```
y_test <- df_test$PRIX
y_pred <- predict(Modele_train, newdata = df_test)
```


On représente ensuite graphiquement les valeurs observées en fonction des valeurs prédites :

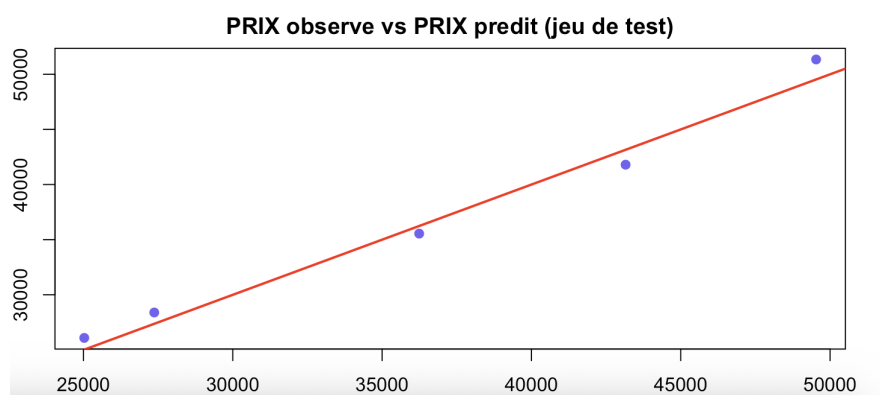


FIGURE 10 – Comparaison entre prix observés et prix prédits sur l'ensemble de test

Sur ce graphique, la droite rouge correspond à la situation idéale où les prédictions sont parfaites ($\hat{Y} = Y$). On observe que les points sont proches de cette droite, ce qui indique une bonne qualité de prédiction.

Métriques obtenues On obtient :

$$R_{\text{test}}^2 = 0.9819.$$

Cela signifie que le modèle explique environ 98% de la variabilité du prix sur des données nouvelles, non utilisées lors de l'estimation. On calcule également l'erreur quadratique moyenne :

```
mean((y_test - y_pred)^2)
```

Ce qui donne :

$$\text{MSE}_{\text{test}} = 1\,552\,658.$$

Cette quantité mesure l'écart moyen entre les valeurs observées et les valeurs prédites sur l'ensemble de test.

Conclusion sur la performance prédictive. Le coefficient R_{test}^2 est très élevé et proche de celui obtenu sur l'ensemble d'apprentissage. De plus, les points du graphique sont bien alignés autour de la première bissectrice. Ces éléments indiquent que le modèle possède une très bonne capacité prédictive sur ce jeu de données. En bref, il ne semble pas y avoir de phénomène de sur-ajustement marqué : le modèle généralise correctement aux observations nouvelles.

Bonne pratique en régression statistique. Dans une démarche de modélisation prédictive, il est recommandé :

- (★₁) d'estimer le modèle sur un ensemble d'apprentissage ;
- (★₂) d'évaluer ses performances sur un ensemble de test ;
- (★₃) de comparer plusieurs modèles possibles ;
- (★₄) et de privilégier un modèle simple offrant de bonnes performances prédictives.

La séparation train-test constitue ainsi une étape essentielle dans toute analyse statistique moderne orientée vers la prédiction.

1.10 Conclusion générale TP1

Cette étude met en évidence plusieurs éléments importants.

- Le jeu de données est de petite taille (18 observations), mais ne présente pas de valeurs manquantes, ce qui permet de travailler dans de bonnes conditions statistiques.
- L'analyse descriptive initiale est indispensable : elle permet de comprendre la structure des données, d'identifier les ordres de grandeur des variables, de détecter d'éventuelles valeurs atypiques et de formuler les premières hypothèses sur les relations entre variables.
- Les visualisations (histogrammes, boxplots, représentations dans des plans de covariables) suggèrent des relations marquées entre certaines variables, et la matrice de corrélation confirme l'existence de dépendances linéaires importantes entre plusieurs caractéristiques des véhicules.
- La représentation des centres de gravité par catégorie de véhicules met en évidence une structuration claire des gammes selon des variables telles que la puissance, la cylindrée et le poids. *On verra plus tard comment apprendre des modèles prédictifs sur des variables catégorielles. C'est fort plaisant.*
- Le modèle de régression linéaire multiple ajusté sur le prix présente un excellent pouvoir explicatif sur l'échantillon, avec un coefficient de détermination R^2 très élevé.
- L'analyse du caractère significatif des coefficients montre que certaines variables jouent un rôle déterminant dans l'explication du prix, tandis que d'autres (*1 seule au seuil 5%*) apparaissent moins informatives, ce qui justifie la construction d'un modèle réduit plus parcimonieux.
- L'étude de la stabilité des coefficients met en évidence leur sensibilité à certaines observations individuelles, ce qui est classique lorsque l'échantillon est de petite taille.
- L'évaluation des performances sur un ensemble de test confirme que le modèle possède une très bonne capacité prédictive et ne présente pas de signe évident de sur-ajustement.
- Enfin, l'étude des résidus ne met pas en évidence de violation flagrante des hypothèses usuelles du modèle linéaire, même si la petite taille de l'échantillon impose de rester prudent dans l'interprétation.

Au-delà des résultats numériques eux-mêmes, ce TP permet surtout de comprendre que la régression linéaire n'est pas simplement une formule à appliquer, mais une démarche complète qui combine exploration des données, modélisation et validation des hypothèses.

lecture des données $\longrightarrow$ analyse descriptive $\longrightarrow$ modélisation $\longrightarrow$ diagnostic du modèle

Retenez bien cette chaîne de raisonnement : elle constitue le cœur de toute analyse statistique appliquée.

Synthèse TP1 : ce qu'il faut absolument savoir faire

Vous devez impérativement retenir et savoir faire les points suivants.

1. Comprendre le jeu de données avant toute modélisation. Avant d'ajuster un modèle, il est indispensable d'examiner les données. Il faut savoir :

- ✓ calculer les statistiques descriptives principales : moyenne, médiane, écart-type ;
- ✓ repérer la position relative des observations par rapport à la médiane ;
- ✓ analyser la distribution des variables à l'aide d'histogrammes (appréhension de la distribution) et de boxplots (détection de valeurs atypiques) ;
- ✓ lire une matrice de corrélation ;
- ✓ identifier un coefficient de corrélation élevé entre deux variables $X^{(j)}$ et $X^{(k)}$, ce qui peut indiquer une dépendance linéaire importante.

Cette étape permet d'anticiper la pertinence d'un modèle linéaire avant même son estimation.

2. Ajuster un modèle de régression linéaire. On souhaite modéliser une relation de la forme

$$Y_i = \beta_0 + \sum_{j=1}^p \beta_j X_i^{(j)} + \varepsilon_i.$$

Sous R, on instancie ce modèle avec la commande

```
lm(Y ~ X1 + ... + Xp).
```

Cette commande construit l'objet modèle contenant toutes les informations nécessaires à l'inférence statistique.

3. Lire correctement la sortie de summary. La commande

```
summary(Modele)
```

est centrale. Il faut absolument savoir interpréter sa sortie. Concernant les coefficients β_j , elle fournit notamment :

- ✓ les coefficients estimés $\hat{\beta}_j$;
- ✓ leurs erreurs standards (mesure de leur imprécision) ;
- ✓ les p-values associées aux tests de significativité individuelle ;

Mais aussi d'autres informations à savoir lire :

- ✓ l'estimation de l'écart-type des résidus $\hat{\sigma}$;
- ✓ le coefficient de détermination R^2 et son jumeau ajusté R_{adj}^2 ;
- ✓ la p-value du test global de Fisher (significativité globale du modèle).

Ces éléments permettent de juger la qualité statistique du modèle.

4. Utiliser les intervalles de confiance des coefficients. La commande

```
confint(Modele)
```

permet d'obtenir des intervalles de confiance pour les coefficients estimés. Elle complète l'analyse des p-values et permet d'évaluer la précision des estimations.

5. Diagnostique des hypothèses sur les résidus. L'étude des résidus constitue une étape essentielle du diagnostic du modèle. Il faut savoir :

- ✓ stocker les résidus `res <- residuals(Modele)` ;
- ✓ tracer l'histogramme des résidus ;
- ✓ vérifier leur centralité (test de Student sur la moyenne) ;
- ✓ examiner leur compatibilité avec l'hypothèse de normalité via le test de Shapiro–Wilk ou encore le QQ-plot.

Ces vérifications permettent de juger si le modèle linéaire est adapté aux données.

6. Produire des prédictions avec le modèle. Une fois le modèle ajusté, on peut estimer la valeur de la variable réponse pour de nouvelles observations avec la commande

`predict(Modele)`.

Cette étape correspond à l'objectif principal d'un modèle de régression : expliquer et prévoir la variable d'intérêt à partir des covariables observées. On dit qu'on *infère* ou *prédit*.